

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Samantha Aguilera Alfaro

11 de septiembre de 2026

21:19

1. Introducción

Este informe compara la teoría con la práctica en dos problemas clásicos: ordenar un arreglo de enteros y multiplicar matrices cuadradas. Del lado del ordenamiento se implementaron Merge Sort, Quick Sort, Patience Sort y Sort; del lado de matrices, el método Naive y el de Strassen. Todos tienen una cota de complejidad bien conocida, pero eso no garantiza que se comporten como uno espera al correrlos de verdad: cosas como el tamaño de la entrada, qué tan ordenados o repetidos están los datos, o simplemente cómo está escrito el código, terminan pesando tanto como la notación asintótica misma. Ese es justamente el punto de este trabajo: medir tiempo y memoria en distintos escenarios y ver dónde la teoría predice bien y dónde no.

2. Entorno experimental

Los experimentos corrieron en un equipo con procesador AMD Ryzen 5 4600H (6 núcleos, 12 hilos, 2.6 GHz), 16 GB de RAM DDR4, Windows 11, GPU NVIDIA GeForce GTX 1650, y se compiló en MSYS2 MINGW64. El código se compiló con g++ 16.1.0. El tiempo se midió con chrono; la memoria, con el pico histórico del *Working Set* de Windows (PeakWorkingSetSize, vía GetProcessMemoryInfo), algo específico de esta plataforma, así que si alguien intenta replicar esto en Linux o macOS esa columna simplemente va a salir en cero.

Para ordenamiento se generaron arreglos de tamaño $n = 10^1, 10^3, 10^5, 10^7$, en orden ascendente, descendente y aleatorio, con dominios $D_1 = \{0, \dots, 9\}$ y $D_7 = \{0, \dots, 10^7\}$, 3 muestras por combinación. Para

matrices, tamaños $n = 2^4, 2^6, 2^8, 2^{10}$, tipos densa/diagonal/dispersa, dominios $D_0 = \{0, 1\}$ y $D_{10} = \{0, \dots, 9\}$, también 3 muestras. Todo se generó con semilla fija para poder repetir los experimentos. Además se puso un límite de 3 minutos por caso, porque sin eso Quick Sort con arreglos ya ordenados simplemente nunca termina (su peor caso $O(n^2)$, que de hecho se activó varias veces en las mediciones).

3. Resultados y Análisis

Los resultados de ordenamiento están en las Fig. 1 y Fig. 2, y los de multiplicación de matrices en las Fig. 3 y Fig. 4. Cada curva es un algoritmo, y los valores están promediados sobre todas las muestras, tipos y dominios. Para mantener el informe conciso, se presentan principalmente los resúmenes generales. Los gráficos desglosados por tipo de entrada y dominio se generaron igualmente y se encuentran disponibles en `data/plots/`.

3.1. Ordenamiento

Lo primero que salta en la Fig. 1 es que Sort gana en prácticamente todos los tamaños, y Quick Sort presenta un rendimiento cercano. mientras el arreglo no sea muy grande. El problema aparece en $n = 10^7$: para arreglos ya ordenados (ascendentes o descendentes), Quick Sort simplemente no terminó dentro del límite de tiempo y esos casos quedaron marcados como omitidos (el detalle está en `quicksort_omitidos.txt`). Con pivote determinista, este es justo el escenario que dispara el peor caso $O(n^2)$ de la teoría —y acá se ve de la forma más contundente posible: ni siquiera alcanza a correr.

Merge Sort, en cambio, se porta bien en todos los tipos de entrada, siempre $O(n \log n)$, aunque un poco más lento que Sort por el costo de ir copiando datos durante la fusión. Patience Sort es el caso más raro de los cuatro: en teoría también es $O(n \log n)$, pero en la práctica queda notoriamente atrás, sobre todo con arreglos aleatorios, por todo el trabajo extra de mantener las pilas y fusionar con un heap de k vías.

En memoria (Fig. 2) las diferencias son bastante claras: Merge Sort reserva un buffer del mismo tamaño que el arreglo ($n \times \text{sizeof}(\text{int})$ bytes), así que es el que más consume. Sort y Quick Sort ordenan *in-place*, por lo que su memoria auxiliar es casi nula (más allá de la pila de recursión). Patience Sort queda en un punto intermedio (nada despreciable, pero lejos del extremo de Merge Sort) consistente con sus estructuras de pilas y el heap de fusión.

3.2. Multiplicación de matrices

En la Fig. 3, Naive crece cúbico como se espera: pendiente 3 en la escala log-log, sin sorpresas. Strassen ($O(n^{2.807})$) muestra una pendiente algo menor a partir de $n = 256$, lo que es consistente con su mejor complejidad asintótica. Pero para $n \leq 64$ pasa lo contrario: Naive es más rápido, porque las sumas y restas extra de Strassen en cada nivel de recursión pesan más que lo que se ahorra en multiplicaciones, un ejemplo bastante directo de que una mejor complejidad asintótica no siempre gana cuando la entrada es chica.

En memoria (Fig. 4) ambos terminan guardando algo parecido para la matriz resultado, pero Strassen necesita espacio extra para las submatrices que arma en cada nivel de recursión, y eso se nota más mientras más grande es n . Naive, al ser puramente iterativo, no requiere estructuras auxiliares adicionales de tamaño superior al de las matrices, por lo que su uso de memoria crece como $O(n^2)$.

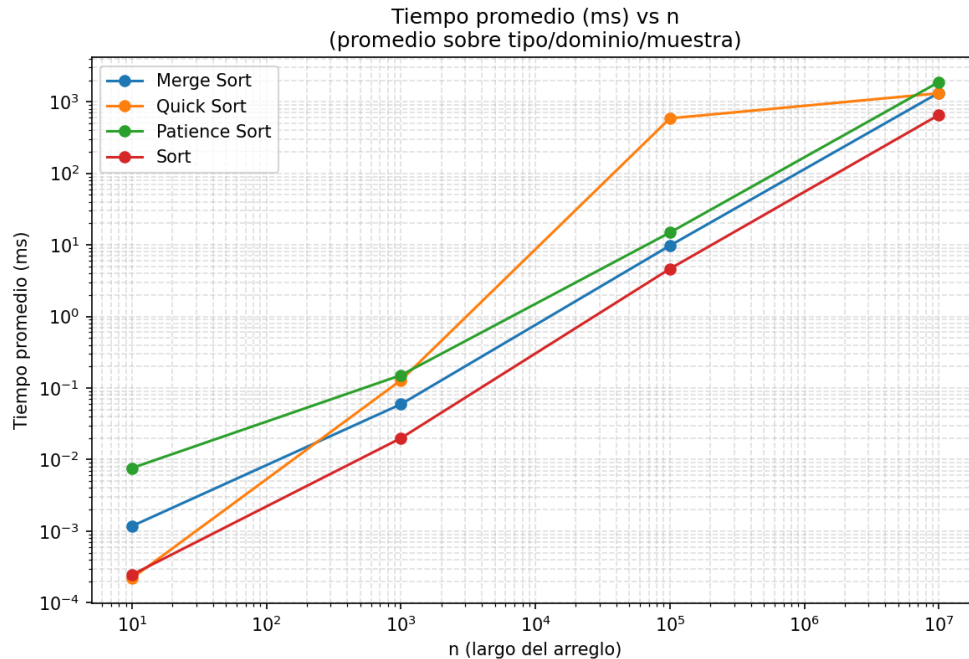


Figura 1: Tiempo de ejecución promedio de los algoritmos de ordenamiento en función de n .

Vale la pena mencionar un par de cosas que no alcanzan a verse en estos cuatro gráficos resumen: Quick Sort resulta mucho más sensible al tipo de entrada que Merge Sort o Sort (que se mantienen estables sin importar si el arreglo viene ordenado, invertido o aleatorio); y en matrices, la ventaja de Strassen se nota más en matrices densas con dominio D_{10} , donde el costo extra de sumar y restar submatrices se diluye mejor. El umbral `STRASSEN_THRESHOLD = 64` de la implementación (ver `strassen.cpp`) explica directamente por qué para $n \leq 64$ el comportamiento es idéntico al de Naive: por debajo de ese tamaño, Strassen es Naive.

En general, la complejidad teórica predice bastante bien el comportamiento asintótico, pero no cuenta toda la historia: la implementación concreta, cómo se maneja la memoria, la localidad de los datos y hasta el tamaño exacto de la entrada terminan siendo igual de importantes en la práctica, sobre todo en el rango de tamaños intermedios donde ninguna curva ha "despegado" todavía.

4. Conclusiones

En general, la complejidad teórica predijo bien el comportamiento general de estos algoritmos, pero no siempre el rendimiento real. Sort fue el más parejo entre tiempo y memoria, sin importar el tipo de entrada; Quick Sort con pivote determinista, en cambio, mostró justo su punto débil clásico: arreglos

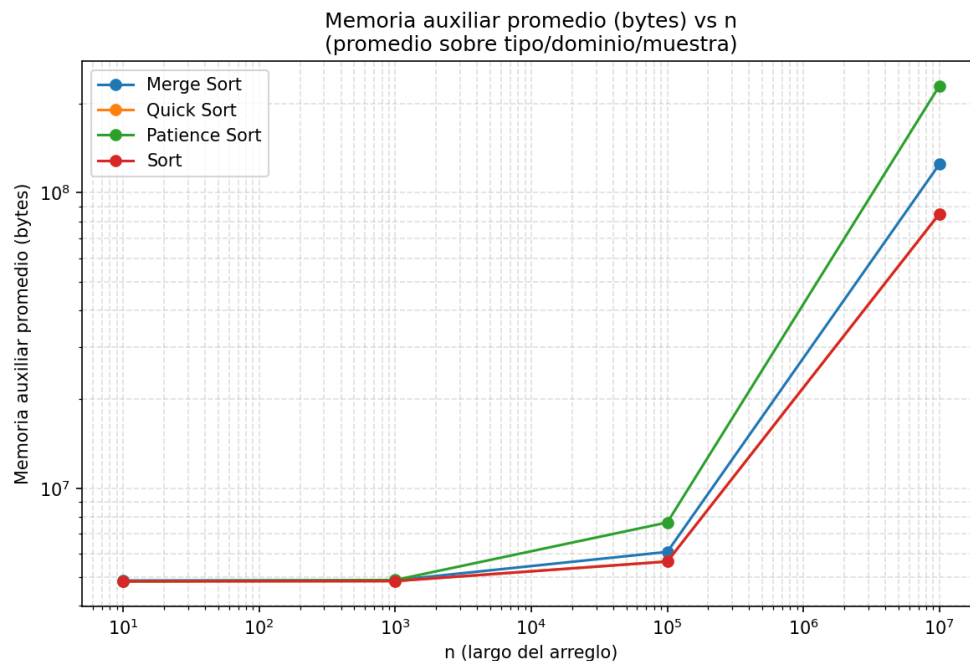


Figura 2: Pico promedio del Working Set utilizado durante la ejecución de los algoritmos de ordenamiento.

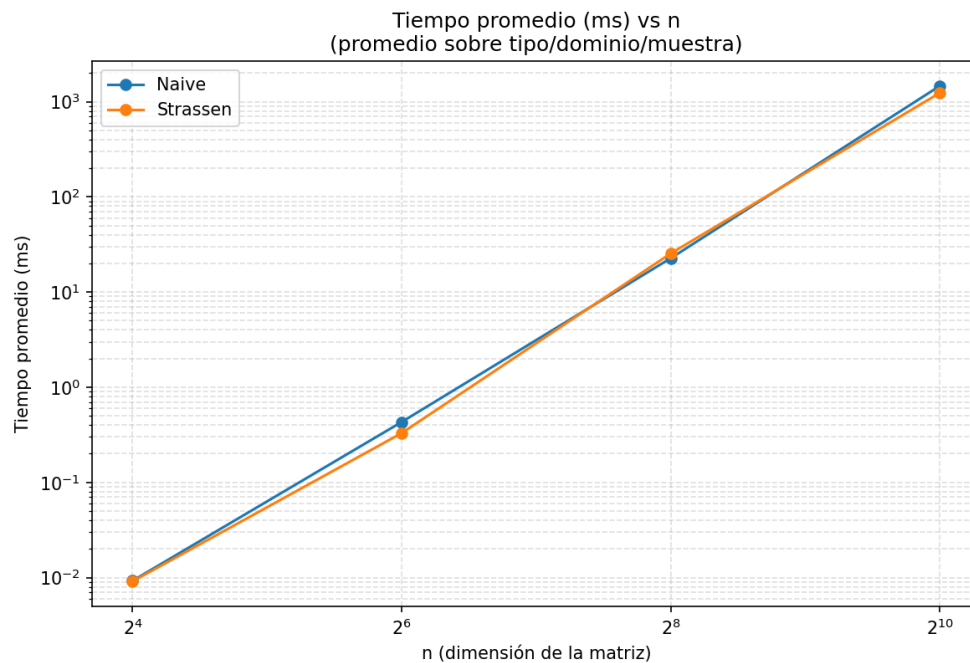


Figura 3: Tiempo de ejecución promedio de los algoritmos de multiplicación de matrices.

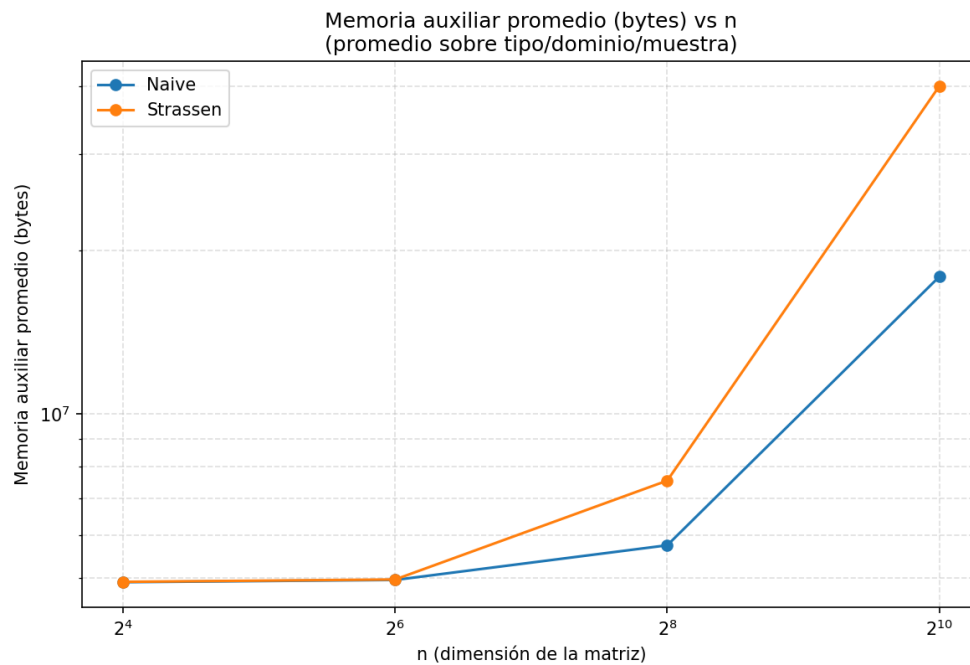


Figura 4: Pico promedio del Working Set utilizado durante la ejecución de los algoritmos de multiplicación de matrices.

ya ordenados, hasta el punto de no terminar dentro del límite de tiempo, confirmando de la forma más directa posible su peor caso $O(n^2)$. Strassen sí rinde más que Naive, pero solo a partir de matrices razonablemente grandes ($n \geq 256$); para las chicas, su propia sobrecarga lo deja atrás. Si algo queda claro de este ejercicio es que la notación asintótica es un buen punto de partida, pero no reemplaza medir: el contexto de uso y las características concretas de los datos importan tanto como la complejidad *en el papel*. En consecuencia, la elección de un algoritmo no debería basarse únicamente en su complejidad asintótica, sino también en las características de los datos, el tamaño de la entrada y los costos asociados a su implementación.